introduces a functionality that wasn't available before CORS; therefore it requires a preflight request.

Upload events are a purely client-side feature: the browser fires the event, and doesn't need to make a server request for more information. But the upload request still requires a preflight because it's introducing functionality that didn't exist before CORS. On the other hand, if you upload a file using `XMLHttpRequest`, but without using upload events, the request doesn't need a preflight (again, this makes sense, because without upload events, the upload behaves the same as a form upload). Table 4.9 shows the distinction between the two types of uploads.

Table 4.9 Uploading with and without upload events. Uploading without upload events doesn't need a preflight.

JavaScript code	Preflight status

`var xhr = new XMLHttpRequest();`
`xhr.open('POST', '/upload', true);`
 	
`xhr.upload.onprogress = function(e) {`
`  console.log('Upload progress: ' ((e.loaded / e.total)`
`* 100));`
`};`
 	
`xhr.send(file);`
	Uses upload events; requires a preflight

`var xhr = new XMLHttpRequest();`
`xhr.open('POST', '/upload', true);`
`xhr.send(file);`
	Doesn't use upload events; no preflight necessary

While this section has done its best to catalog all the cases where a preflight request is made, there may be new types of requests in the future that issue a preflight. But as long as your server is armed with the Access-Control-Allow-Methods and Access-Control-Allow-Headers headers, you'll be prepared to answer any preflight request that comes your way.

4.6 Preflight result cache

One downside of the preflight request is that it issues two HTTP requests, one for the preflight and a second for the actual request. This can be a performance concern because HTTP requests are expensive, especially on resource-constrained devices like smartphones. To help reduce the number of preflight requests, preflight responses can be cached in a preflight result cache.

The preflight result cache stores the responses to a preflight request for a particular URL. If a request is made to the same URL, the browser first checks the preflight result cache to see if there is already a response. If the browser finds a response in the cache, it skips the preflight request and goes straight to the actual request. If there is no response in the cache, the browser sends the preflight, and then stores the response of that preflight in the cache. Figure 4.21 shows how responses flow through the preflight result cache.